

DATA DRIVEN PRODUCT RECOMMENDATION SYSTEM

A PROJECT REPORT

Submitted by

SANJAY S.S (962822205034)

NEELAKANDAN S (962822205301)

ABHILASH A.S (962822205304)

BEZALEL VIVIAN SALES S (962822205306)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY



UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

ANNA UNIVERSITY :: CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY :: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**PRODUCT RECOMMENDATION SYSTEM**” is the Bonafide work of “**Sanjay S.S (962822205034), Neelakandan S (962822205301), Abhilash A.S (962822205304), Bezalel Vivian Sales S (962822205306)**” who carried out the project work under my supervision.

Dr. A. Radhakrishnan, M.E., Ph.D., Dr. J. Vijla, MTech., Ph.D.,

HEAD OF THE DEPARTMENT

SUPERVISOR,

Assistant Professor (Sl.Gr),

Assistant Professor (Sl.Gr),

Information Technology,

Information Technology,

University College of Engineering,

University College of Engineering,

Nagercoil, Konam - 629 004

Nagercoil, Konam - 629 004

PROJECT TEAM

Sanjay S.S
Neelakandan S
Abhilash A.S
Bezalel Vivian Sales S

Academic Year : 2025-2026

ABSTRACT

The rapid growth of e-commerce has made product discovery a significant challenge, particularly for new users facing the "cold-start" problem. To address this, we present the Data-Driven Product Recommendation System (DDPRS), an intelligent, highly scalable architecture designed to maximize user engagement through real-time behavioral telemetry and deep learning. The system continuously captures implicit user actions—such as clicks, cart additions, and hover durations—and processes them through a Neural Collaborative Filtering (NCF) model to predict personalized item engagement. To seamlessly handle new users with no prior history, DDPRS implements a hybrid cold-start module utilizing Maximal Marginal Relevance (MMR). This algorithm enforces a "Diversity Tax" via a 70/30 Exploit-Explore strategy, balancing globally trending items with category diversity. By integrating deep latent feature extraction with intelligent cold-start heuristics, DDPRS significantly enhances product discovery and provides a highly optimized, personalized shopping experience for all users.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who have contributed to the successful completion of this project. First and foremost, we thank the Almighty for providing us with the strength, knowledge, and perseverance to complete this project successfully.

We express our heartfelt thanks to our honorable and respected **DEAN, Dr. V. A. Nagarajan, Ph.D.**, for his continuous support and for fostering a conducive academic environment throughout our course of study.

We extend our deep sense of gratitude to our **HEAD OF THE DEPARTMENT, Dr.A.RADHAKRISHNAN.,M.E., Ph.D.**, Assistant Professor, of Information Technology, for his valuable guidance, encouragement, and support during the course of this project.

We are highly thankful to our **Project Coordinator and Supervisor, Dr. J. Vijila, M.Tech., Ph.D.**, Assistant Professor, Department of Information Technology, for her constant encouragement, insightful suggestions, valuable guidance, continuous support, and constructive feedback that greatly contributed to the successful completion of this project.

We extend our thanks to all the **teaching and non-teaching staff members** of the department for their cooperation and support. Finally, we express our deepest gratitude to our parents and family members for their unwavering support, encouragement, and blessings, which played a vital role in the successful completion of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	Abstract	iii
	Acknowledgement	iv
	List of figures	x
1	Introduction	1
	1.1 OVERVIEW	1
	1.2 Scope of the Project	1
	1.3 Problem Statement	2
	1.4 Purpose	2
	1.5 Scope	2
	1.6 Contribution	3
2	Literature Review	4
	2.1 Intelligent classification and personalized recommendation	4
	2.2 Harnessing multimodal data and multi- recall strategies	5
	2.3 A neural matrix decomposition recommender system	6
	2.4 Knowledge graph-based recommendation system	6
	2.5 Leveraging large language models to enhance personalized	7

2.6	Unveiling insights from Customer reviews	8
2.7	Applying machine learning Algorithm to optimize	9
2.8	Dominance of ai and machine Learning techniques in hybrid movie recommendation system	9
2.9	Personalized recommendation Systems powered by large language models	10
2.10	Digital media recommendation System design based on user behavior analysis and emotional feature extraction	11
3	System Analysis	13
3.1	Problem definition	13
3.2	Existing system	13
3.2.1	Disadvantages	14
3.3	Proposed system	14
3.3.1	Advantages	15
4	System requirements	16
4.1	Hardware requirements	16
4.2	Software requirements	16
4.3	Software description	16

5	System Design	18
5.1	System architecture	18
5.1.1	User interaction and Telemetry management	18
5.1.2	Deep learning and product Recommendation	19
5.1.3	Asynchronous processing and Monitoring	19
5.1.4	Database and system Integration	19
5.2	Use case diagram	20
5.3	Class diagram	21
5.4	Activity diagram	21
5.5	Sequence diagram	23
6	System Implementation	24
6.1	Modules of the project	24
6.1.1	Catalog management module	24
6.1.2	Real-time telemetry module	26
6.1.3	Cold start module	27
6.1.4	Neural collaborative filtering Module	29
6.1.5	Backend api module	30
7	Experimental Results	31
7.1	Catalog browsing Screen	31

	7.2 Backend data logging	32
	And scoring	
	7.3 Recommendation	33
	Screens	
8	Results And Findings	35
	8.1 Performance evaluation	35
	8.1.1 response time comparison	35
	8.1.2 Recommendation accuracy	36
	Comparison	
	8.1.3 Cold start user engagement	36
	Comparison	
9	Conclusion And Future	38
	Enhancement	
	9.1 Conclusion	38
	9.2 Future enhancements	38
	Appendix	xii
	References	xv
	Certificate	xvii

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
5.1	System Architecture	18
5.2	Use Case Diagram	20
5.3	Class Diagram	21
5.4	Activity Diagram	22
5.5	Sequence Diagram	23
6.1	Catalog Management Module Flow	25
6.2	Real-Time Telemetry Module Flow	26
6.3	Cold Start Module Flow	28
6.4	Neural Collaborative Filtering Module Flow	29
6.5	Backend API Module Flow	30
7.1	Home Dashboard Screen	31
7.2	Category Filter Screen	31
7.3	Interaction Telemetry Table	32
7.4	Global Product Catalog	33
7.5	Personalized Recommendations	34

7.6	Cold Start Popularity Screen	34
8.1	Response Time Comparison	35
8.2	Recommendation Accuracy Comparison	36
8.3	Cold Start Engagement Comparison	37

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
API	-	Application Programming Interface
CSV	-	Comma-Separated Values
GBR	-	Gradient Boosting Regressor
HTML	-	Hyper Text Markup Language
ML	-	Machine Learning
PRS	-	Product Recommendation System
EDA	-	Exploratory Data Analysis
SVD	-	Singular Value Decomposition

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Recommendation systems enhance personalized shopping experiences in modern e-commerce platforms. However, cold start affects recommendation accuracy during initial user engagement stages. Data-driven approaches analyze user behavior and product attributes for suggestions. Analytical and hybrid methods support meaningful early recommendations. Systems progressively improve recommendations as user interactions increase over time.

The proposed DDPRS addresses these challenges by implementing interactive user onboarding to capture initial preferences. The system employs content-based filtering using similarity measures such as cosine similarity, collaborative filtering, and hybrid approaches to generate accurate product recommendation. By incorporating numerical encoding of user preferences and knowledge transfer from past cold start users, the system significantly improves early-stage personalization and enhances the overall user experience.

1.2 SCOPE OF THE PROJECT

The primary objective of this project is to design and implement an AI-powered DDPRS that resolves the cold start problem and provides highly personalized product suggestions. Specific goals include developing a universal dataset adapter for flexible data ingestion, implementing a behavioral cold start mechanism that passively tracks user interactions, integrating multiple recommendation algorithms including collaborative and content-based filtering, and creating a weighted ensemble model to combine predictions from different algorithms for improved accuracy. Furthermore, the system aims to provide explainable product recommendations by showing users why specific products

are suggested, and to offer a production-ready REST API for easy integration with e-commerce platforms.

1.1 PROBLEM STATEMENT

Traditional recommendation systems struggle significantly with new users who have no purchase or rating history, a situation known as the cold start problem. This leads to generic, non-personalized product recommendations, poor user engagement, and lost revenue opportunities. Existing solutions often rely on lengthy questionnaires or limited demographic data, which create friction and provide only superficial personalization. The problem is to generate accurate recommendations for users with no history, capture user preferences at first interaction, and improve early-stage personalization.

1.2 PURPOSE

The primary purpose of this study is to develop an intelligent DDPRS that generates accurate recommendations for users with no historical data. The system captures user preferences at the first interaction using interactive onboarding and improves early-stage personalization through hybrid recommendation models and knowledge transfer from past cold start users.

1.3 SCOPE

The study focuses on developing a machine learning-based framework for personalized product recommendations. It involves the design and implementation of user registration and preference collection, product management, data preprocessing and EDA, a hybrid recommendation engine using content-based and collaborative filtering, and a feedback mechanism for continuous improvement.

1.3.3 Contribution of the Project

Hybrid Recommendation Architecture: The project presents a unified, intelligent system that seamlessly switches between a deep learning Neural

Collaborative Filtering (NCF) model for established users and a dynamic heuristic scoring engine for new users, ensuring continuous and highly accurate personalization.

Novel Cold-Start Resolution: The system utilizes a zero-friction behavioral learning mechanism to solve the e-commerce Cold Start problem. By instantly analyzing a new user's very first interactions via dynamic SQL weighting, it builds a personalized product feed immediately without requiring intrusive onboarding questionnaires.

Asynchronous Telemetry Tracking: The implementation of a lightweight "Invisible Notebook" telemetry logger captures critical user interactions (clicks and views) in real-time. It utilizes background POST requests to continuously enrich the SQLite database without ever impacting frontend rendering or page load speeds.

Filter Bubble Mitigation: The system addresses the industry-standard "over-specialization" problem by incorporating the Maximal Marginal Relevance (MMR) algorithm. By applying a dynamic "diversity tax" during candidate ranking, it ensures users are exposed to a diverse catalog rather than being trapped in a single-category loop.

CHAPTER 2

LITERATURE REVIEW

The literature review explores various machine learning and AI-driven methodologies for building product recommendation systems. Recent research focuses on addressing key challenges such as the cold-start problem, personalization accuracy, and scalability. Various techniques including collaborative filtering, content-based filtering, knowledge graph embedding, deep neural networks, and large language models have been employed to improve recommendation quality. Studies highlight the importance of multimodal data integration and hybrid approaches for enhanced personalization.

2.1 Intelligent Classification and Personalized Recommendation (2024)

Kangming Xu et.al [1] represent a ML-based personalized recommendation system designed to analyse user behaviour pattern and classify products for generating personalized suggestions. The study involved analysing user interaction data from an e-commerce platform to evaluate recommendation performance. Using ML models, the system achieved significant improvements in shopping efficiency and customer satisfaction compared to baseline methods. The paper highlights the technical advantages of using user behaviour analysis and classification algorithms to provide relevant, automated recommendations.

Advantages:

- Machine learning models provide improved accuracy in recognizing user preferences across different product categories.
- User behavior analysis eliminates the need for manual preference collection and allows the system to learn complex patterns effectively.
- Capable of handling large-scale user interaction data and improving classification performance using ML algorithms.

Disadvantages:

- Privacy concerns arise due to collection of user behavior data for recommendation purposes.
- Cold start problem persists for new users with no interaction history.
- Performance decreases for new products with limited user feedback.

2.2 Harnessing Multimodal Data and Multi- Recall Strategies (2024)

Siyue Li et al. [2] introduces a sophisticated recommendation framework designed to harness multimodal data and multi-recall strategies for enhanced product discovery. The system is composed of three key components: a multimodal data processor that handles text, images, and user reviews; a multi-recall generator that retrieves candidate items through diverse channels; and a Light-GBM ranker that orders the retrieved items by relevance. The paper highlights the technical advantages of using multiple data modalities and recall strategies to provide comprehensive, accurate recommendations.

Advantages:

- Provides detailed analysis of multimodal data which is essential for accurate product understanding.
- Supports multiple recall strategies for improved product discovery and recommendation.
- Improves recommendation precision through effective ranking techniques.

Disadvantages:

- High computational requirements due to processing of multiple data streams simultaneously.
- Performance varies based on data quality and availability.
- Increased system complexity and implementation difficulty.

2.3 A Neural Matrix Decomposition Recommender System (2024)

Ao Xiang et al. [3] propose a novel neural matrix decomposition approach integrated with multimodal large language models for capturing complex user-item interactions. The system includes a multimodal LLM encoder, a neural matrix decomposition module, and a prediction engine for personalized recommendations. Using neural matrix decomposition with LLM integration, the system achieved better user-item interaction modelling compared to traditional matrix factorization methods. The paper highlights the technical advantages of using LLMs to enhance latent factor representations for improved recommendation accuracy.

Advantages:

- Deep learning techniques improve product recommendation by learning complex patterns from user-item interactions.
- Provides better generalization and accuracy compared to traditional machine learning approaches.
- Capable of handling large datasets and extracting meaningful preference features efficiently.

Disadvantages:

- Requires high computational power and memory for training deep learning models.
- Complex training requirements due to large model size.
- Needs large annotated datasets which may not always be available.

2.4 Knowledge graph-based recommendation system (2023)

Zeinab Shokrzadeh et al. [4] introduce a knowledge graph-based recommendation system leveraging semantic relationships between users, products, and attributes. Using knowledge graph embeddings with neural collaborative filtering, the system achieved higher recall and precision

compared to standard collaborative filtering methods. The paper highlights the technical advantages of using semantic relationships to improve recommendation quality and overcome data sparsity limitations.

Advantages:

- Provides a comprehensive framework for understanding user-product relationships using knowledge graphs.
- Highlights importance of semantic relationships in improving recommendation accuracy.
- Covers multiple approaches including collaborative filtering and graph-based techniques.

Disadvantages:

- Knowledge graph construction is complex and requires significant domain expertise.
- Limited focus on real-time implementation and practical deployment challenges.
- Requires high computational resources for graph processing.

2.5 Leveraging Large Language Models to Enhance Personalized Recommendations (2024)

Wei Xu et.al [5] present a large language model-based recommendation system that uses deep semantic understanding to generate personalized product suggestions. The system includes an LLM fine-tuning module, contextual understanding engine, and recommendation generator. The study involved fine-tuning LLMs on product catalogs and user interaction data from major e-commerce platforms. Using LLM-based recommendation, the system achieved improvements in both precision and diversity of recommendations compared to traditional methods. The paper highlights the technical advantages of using LLMs for deep semantic understanding and contextual recommendation generation.

Advantages:

- Deep learning techniques improve recommendation accuracy by learning complex semantic patterns.
- Provides better diversity and personalization compared to traditional approaches.
- Capable of generating explainable recommendations with contextual understanding.

Disadvantages:

- Requires large training datasets and substantial computing resources.
- High computational cost for model deployment and inference.
- Performance depends on quality and size of training data.

2.6 Unveiling Insights from Customer Reviews (2024)

Zhiming Zhao et al. [6] introduce a hybrid recommendation architecture combining BERT with deep neural networks for enhanced semantic understanding. The system includes a BERT encoder, DNN feature extractor, and fusion module. Using the transformer-DNN fusion architecture, the system achieved better semantic understanding of product content compared to traditional text processing methods. The paper highlights the technical advantages of using transformer-based models for capturing nuanced textual semantics in recommendation tasks.

Advantages:

- Transformers provide superior semantic understanding of products.
- Fusion architecture captures both contextual and deep features.
- Enhanced text analysis directly improves recommendation accuracy.

Disadvantages:

- High processing complexity due to transformer and DNN integration.
- Requires significant computational resources for training and inference.

- Limited scalability for real-time applications.

2.7 Applying Machine Learning Algorithm to Optimize (2024)

Wangmei Chen et al. [7] present a collaborative filtering-based recommendation system enhanced with optimization algorithms for personalized education recommendations. The system includes a user preference collector, collaborative filtering engine, and optimization module. The study involved testing the system on educational datasets containing student interactions with learning resources. Using collaborative filtering with optimization algorithms, the system achieved improvements in personalized learning experiences compared to standard recommendation methods. The paper highlights the technical advantages of using optimization techniques to enhance collaborative filtering performance in sparse data environments.

Advantages:

- Collaborative filtering with optimization improves personalization in sparse data environments.
- Supports adaptive learning pathways based on user progress.
- Provides better handling of data sparsity compared to standard methods.

Disadvantages:

- Affected by sparse data which limits recommendation quality for new items.
- Performance varies based on availability of user interaction data.
- Scalability challenges for large user bases.

2.8 Dominance of AI and Machine Learning Techniques in Hybrid Movie Recommendation System (2024)

MD Rokibul Hasan et al. [8] present a hybrid movie recommendation system that integrates both text-to-number conversion and cosine similarity

approaches to predict the most top-rated and desired movies for targeted users. The system employs the Alternating Least Squares (ALS) algorithm to reinforce the overall accuracy of the recommendations. The study involved performing rigorous evaluations using the "TMDB 5000 Movie Dataset" from Kaggle. By running experiments that categorized the dataset into distinct modules, the system achieved a Root Mean Square Error (RMSE) of 0.8252 and a final accuracy of approximately 99.8%. The paper highlights the technical advantages of combining ALS algorithms with text-to-number conversion to significantly outperform existing state-of-the-art recommendation models.

Advantages:

- Integrates text-to-number conversion and cosine similarity to provide highly accurate and relevant user recommendations.
- The use of the Alternating Least Squares (ALS) algorithm significantly reinforces the precision of the predictions.
- Achieves an exceptionally high accuracy rate (nearly 99.8%) compared to existing baseline models.

Disadvantages:

- Text-to-number conversion and hybrid processing can require high computational resources when scaled to massive datasets.
- The system's high performance is heavily dependent on the specific structure and quality of the TMDB dataset.
- The complex integration of ALS and cosine similarity may introduce latency challenges for real-time, instantaneous personalization.

2.9 Personalized Recommendation Systems Powered By Large Language Models (2024)

Fu Shang et al. [9] propose a novel personalized recommendation system that leverages Large Language Models (LLMs) to seamlessly integrate deep semantic understanding with user preferences. The proposed system addresses

the critical challenges and limitations found in traditional recommendation approaches by harnessing the advanced natural language processing capabilities inherent in LLMs. By analyzing complex textual data, the framework effectively bridges the gap between nuanced user intent and specific product features. The study demonstrates that utilizing large language models significantly improves the overall precision, diversity, and contextual relevance of personalized product suggestions.

Advantages:

- Leverages the advanced natural language processing capabilities of LLMs to achieve a deep semantic understanding of user preferences.
- Successfully overcomes critical challenges present in traditional recommendation systems by integrating rich textual context.
- Significantly improves the overall precision, relevance, and personalization of the generated recommendations.

Disadvantages:

- Deploying and querying Large Language Models requires substantial computational resources, which increase operational costs.
- Real-time inference with complex LLMs can introduce latency issues, potentially slowing down the instantaneous user browsing experience.
- The system's effectiveness relies heavily on the quality, structure, and volume of the textual data used to prompt or fine-tune the model.

2.10 Digital Media Recommendation System Design Based on User Behavior Analysis and Emotional Feature Extraction (2025)

Ting Ruan et al. [10] propose a digital media recommendation system designed to deeply integrate user behavior analysis with advanced emotional feature extraction. The proposed framework aims to capture a more holistic view

of user preferences by bridging traditional interaction tracking. The study involved rigorous training and evaluation using the large-scale MovieLens 20M dataset. By successfully fusing these emotional contexts with standard behavioral data, the system achieved significant improvements in recommendation relevance and overall accuracy compared to baseline collaborative filtering methods. The paper highlights the technical advantages of utilizing multi-dimensional feature extraction to better understand and predict the sentiment-driven choices of users in modern digital media platforms.

Advantages:

- Integrates emotional features with user behavior for deeper, sentiment-aware personalization.
- Evaluated on the large-scale MovieLens 20M dataset, demonstrating high reliability.
- Overcomes collaborative filtering "blind spots" by factoring in emotional context.

Disadvantages:

- Extracting emotional features requires heavy NLP pipelines, increasing computational overhead.
- Emotional analysis accuracy relies heavily on the quality and length of user reviews.
- Fusing behavioral and emotional datasets may introduce latency.

CHAPTER 3

SYSTEM ANALYSIS

3.1 PROBLEM DEFINITION

Delivering a highly personalized shopping experience is a significant issue in modern e-commerce, particularly for platforms struggling to retain users who demand immediate relevance. Due to cold start problems and lack of data, systems often forget to tailor suggestions, skip important behavioral cues, or fail to follow user preferences accurately. In many cases, platforms are unable to maintain a consistent personalization schedule because of long inference times, sparse data, or lack of proper tracking.

This leads to:

- Reduced effectiveness of recommendations
- Increased drop-off rates
- Poor user engagement

3.2 EXISTING SYSTEM

In existing e-commerce platforms, product recommendation systems primarily rely on basic keyword searches and simple collaborative filtering. Users often browse catalogs without receiving personalized suggestions that match their actual, real-time interests. These systems do not provide intelligent product recommendations based on deep behavioral analysis. Most existing platforms rely on static algorithms that fail to adapt when a new user registers, leading to a severe "cold-start" problem where no products can be suggested. Platforms also face challenges when handling heavy ML computations, as traditional synchronous architectures slow down the web interface and cause lag. They must rely on overnight batch processing to update suggestions, which is time-consuming and inefficient. Many existing platforms also lack proper

mechanisms to introduce product diversity, trapping users in "filter bubbles" where they only see identical items. Due to this limitation, platforms overlook cross-category sales opportunities. As a result, the current recommendation process often leads to poor user engagement and a stagnant shopping experience.

3.2.1 Disadvantages

- Lack of real-time, intelligent product recommendation models.
- Difficulty in providing suggestions for brand new users (Cold-Start problem).
- Synchronous web architectures cause UI lag during heavy data processing.
- High possibility of trapping users in narrow filter bubbles.
- Limited support for discovering diverse, cross-category products.
- Platforms rely on inefficient, outdated batch processing for updates.
- Users may miss relevant products due to poor algorithmic matching.

3.3 PROPOSED SYSTEM

The proposed system introduces a Data-Driven Product Recommendation System (DDPRS) that improves the e-commerce experience by using deep learning technologies. The system analyzes real-time user telemetry data, such as clicks and views, to identify behavioral patterns and recommend highly relevant products. Neural Collaborative Filtering (NCF) techniques are used to extract deep, non-linear relationships from the interaction logs. Based on this analysis, the system calculates an accurate engagement prediction score for unseen items. The system also utilizes an intelligent cold-start mechanism that instantly provides popular, trending products to new users without interaction

history. In addition, the system performs real-time asynchronous processing using FastAPI and Uvicorn, which ensures zero-latency catalog browsing. It

provides an invisible background logging mechanism that securely stores interaction history in an SQLite database. E-commerce platforms can use this modular system to seamlessly track user activity, manage catalogs, and dynamically serve AI-driven suggestions without any performance bottlenecks.

3.3.1 Advantages

- Intelligent product recommendations powered by Neural Collaborative Filtering.
- Automatic and silent user telemetry tracking in real-time.
- Instant cold-start resolution for brand new users.
- Zero-latency web browsing through an asynchronous FastAPI backend.
- Improved e-commerce engagement through accurate cross-category matching.
- Modular and stable execution environment preventing OS-level conflicts.
- Highly efficient database management using SQLite for secure interaction logging.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

- **Processor:** Intel i5 (12th Gen or higher) or Intel i7
- **RAM:** Minimum 16 GB (Crucial for NCF model inference)
- **Storage:** At least 512 GB NVMe SSD for fast data retrieval
- **Display:** Standard Laptop screen or 19" Monitor
- **Internet Connection:** Required for server communication and real-time features.

4.2 SOFTWARE REQUIREMENTS

- **Operating System:** Windows 11 /Linux/macOS
- **Programming Languages:** Python 3.11, JavaScript (ES6)
- **Frontend Technologies:** HTML5, CSS3, Vanilla JavaScript
- **Backend Framework:** FastAPI (served via Uvicorn ASGI Server)
- **Database Management:** SQLite
- **Deep Learning & Data Libraries:** TensorFlow, Keras, Pandas, NumPy
- **IDE / Editor:** Visual Studio Code (VS Code)

4.3 SOFTWARE DESCRIPTION

- **Core Execution Environment (Python 3.11) :** Python serves as the primary programming language for the project, seamlessly integrating the high-speed web server with the heavy machine learning computations.
- **Frontend Technologies (HTML, CSS, Vanilla JavaScript) :** HTML, CSS, and Vanilla JavaScript construct the responsive user interface. JavaScript specifically manages asynchronous API requests and silently tracks behavioral telemetry in the background.

- **Backend Framework (FastAPI & Uvicorn) :** FastAPI and Uvicorn are used to build the asynchronous RESTful API. They handle user requests and machine learning inference, ensuring zero-latency communication without blocking the user interface.
- **Database Management (SQLite) :** SQLite is a lightweight, serverless database used to securely store product catalogs, category mappings, and real-time user telemetry logs for fast data retrieval.
- **Machine Learning Ecosystem (TensorFlow, Keras & Pandas) :** These libraries handle the data preprocessing and deep learning architecture. They are strictly utilized to convert telemetry logs into tensors and train the Neural Collaborative Filtering (NCF) model.

CHAPTER 5

SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The DDPRS follows a layered, multi-tier architecture consisting of the presentation layer, application layer, machine learning layer, and data layer mention in Fig. No. 5.1. The presentation layer is developed using HTML5, CSS3, and Vanilla JavaScript to provide a simple, responsive, and dynamic interface for users to interact with the e-commerce platform. Shoppers can browse catalogs, apply category filters, and view personalized product dashboards through this interface. It includes advanced features like an intelligent cold-start mechanism and real-time behavioral telemetry tracking. The machine learning layer utilizes TensorFlow and Keras to process this telemetry, running the Neural Collaborative Filtering (NCF) model to connect suitable products to users based on deep behavioral patterns

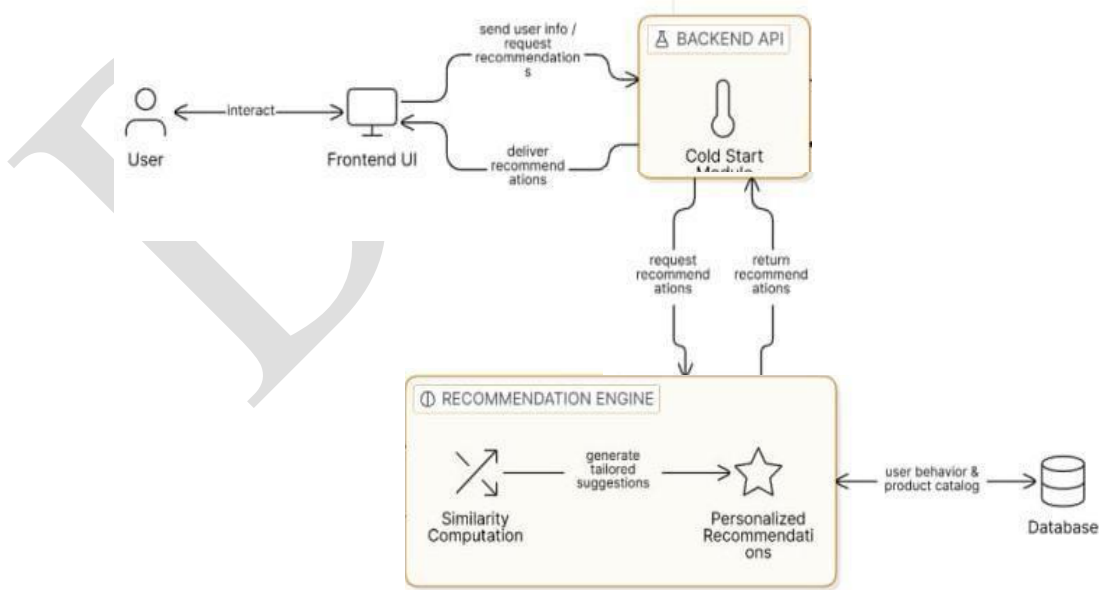


Fig. No. 5.1 System Architecture

5.1.1 User Interaction and Telemetry Management

Shoppers access the e-commerce platform through a responsive frontend interface to browse product catalogs. The system dynamically adapts based on history: returning users receive deeply personalized grids, while new users are routed to trending items. Throughout this interaction, the system silently captures behavioral telemetry (clicks and views) to ensure continuous learning and seamless data tracking.

5.1.2 Deep Learning and Product Recommendation

To support real-time product discovery, the system passes user interaction logs to the deep learning engine. The Neural Collaborative Filtering (NCF) model computes complex, non-linear patterns to identify the most relevant products for returning shoppers. By continuously updating its weights based on new session data, the model's prediction accuracy naturally improves. For users without history, an intelligent cold-start heuristic instantly manages product discovery using global popularity metrics, ensuring highly efficient personalization.

5.1.3 Asynchronous Processing and Monitoring

Machine Learning computations are resource-intensive, the FastAPI asynchronous backend ensures zero-latency catalog browsing. The frontend UI never freezes while the NCF model runs in the background. Meanwhile, the system actively monitors asynchronous POST requests, securely logging interaction data without ever interrupting the shopper's experience.

5.1.4 Database and System Integration

All system data, including product catalogs, category mappings, and user telemetry, is securely stored in a centralized SQLite database. By seamlessly integrating the Vanilla JavaScript frontend, FastAPI backend, and TensorFlow

engine, this architecture ensures smooth communication and efficient data management, ultimately improving overall system performance and scalability.

5.2 USE CASE DIAGRAM

Fig. No. 5.2 explains the primary interactions between the User and the System. User Actions: The user can browse the product catalog, view specific product details, and access their dynamically generated "For You" recommendations. System Actions: The backend system automatically performs background actions such as logging telemetry via POST requests, triggering the NCF algorithms, and returning sorted JSON data without interrupting the user's session.

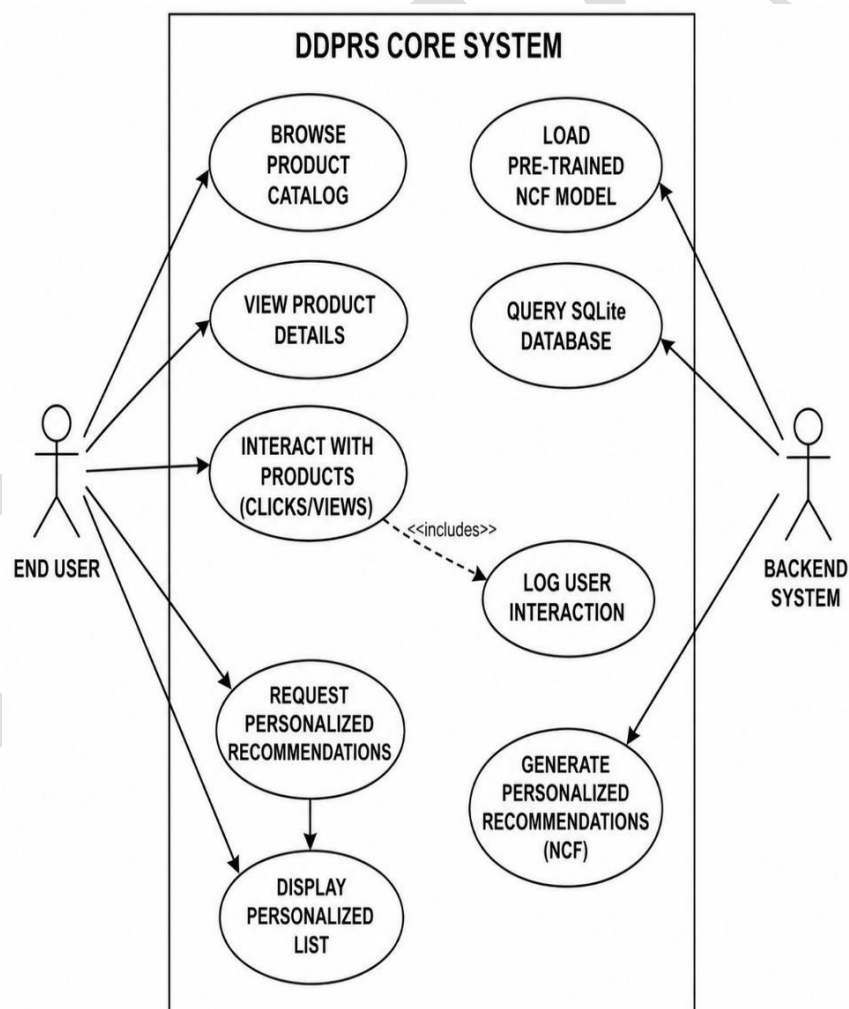


Fig. No. 5.2 Use Case Diagram

5.3 CLASS DIAGRAM

Fig. No. 5.3 outlines the core objects and their relationships. Key classes include: API Router: Handles endpoints like /products, /categories, and /tracking/behavior. Models/Schemas (Pydantic): Defines the data structures required for user interactions and product catalogs. NCFRecommender: The core ML class containing methods like load() to ingest the .h5 file and predict() to run inference on user-item pairs.

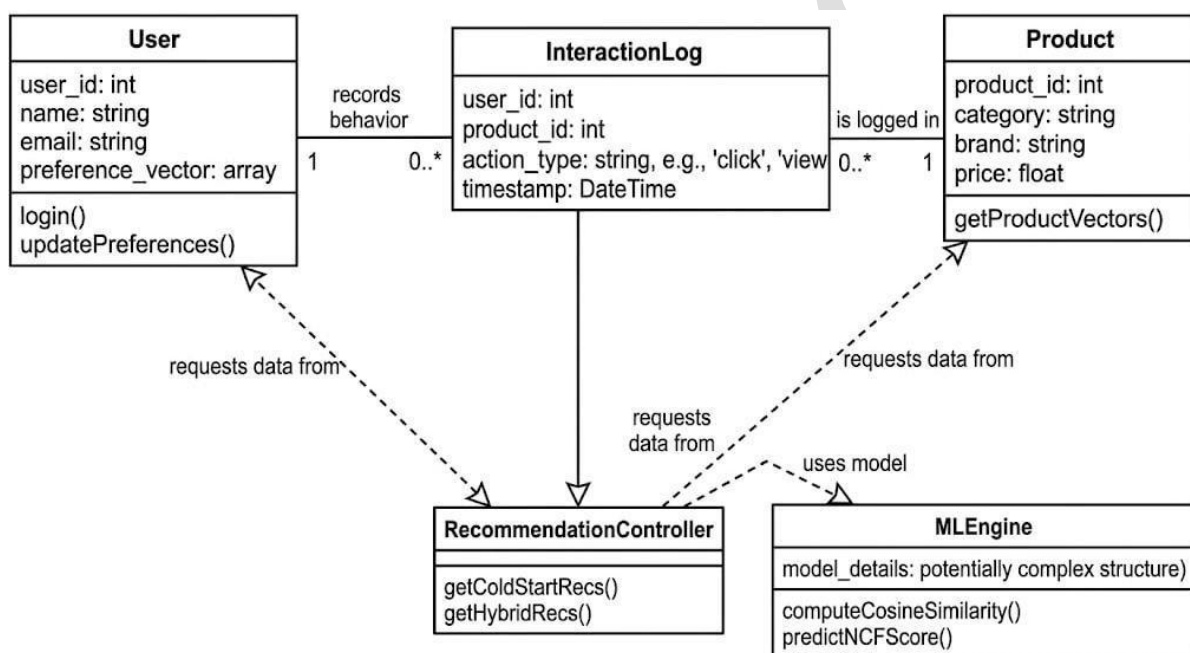


Fig. No. 5.3 Class Diagram

5.4 ACTIVITY DIAGRAM

Fig. No. 5.4 details the step-by-step workflow. The flow begins with the user opening the application. The system checks for interaction history. If none exists, the Cold Start path queries the database for top-trending items. As the user clicks and browses, Vanilla JS sends asynchronous tracking payloads to the FastAPI backend. This data is logged in SQLite. On subsequent refreshes, the backend pulls this new telemetry, feeds it into the NCF model.

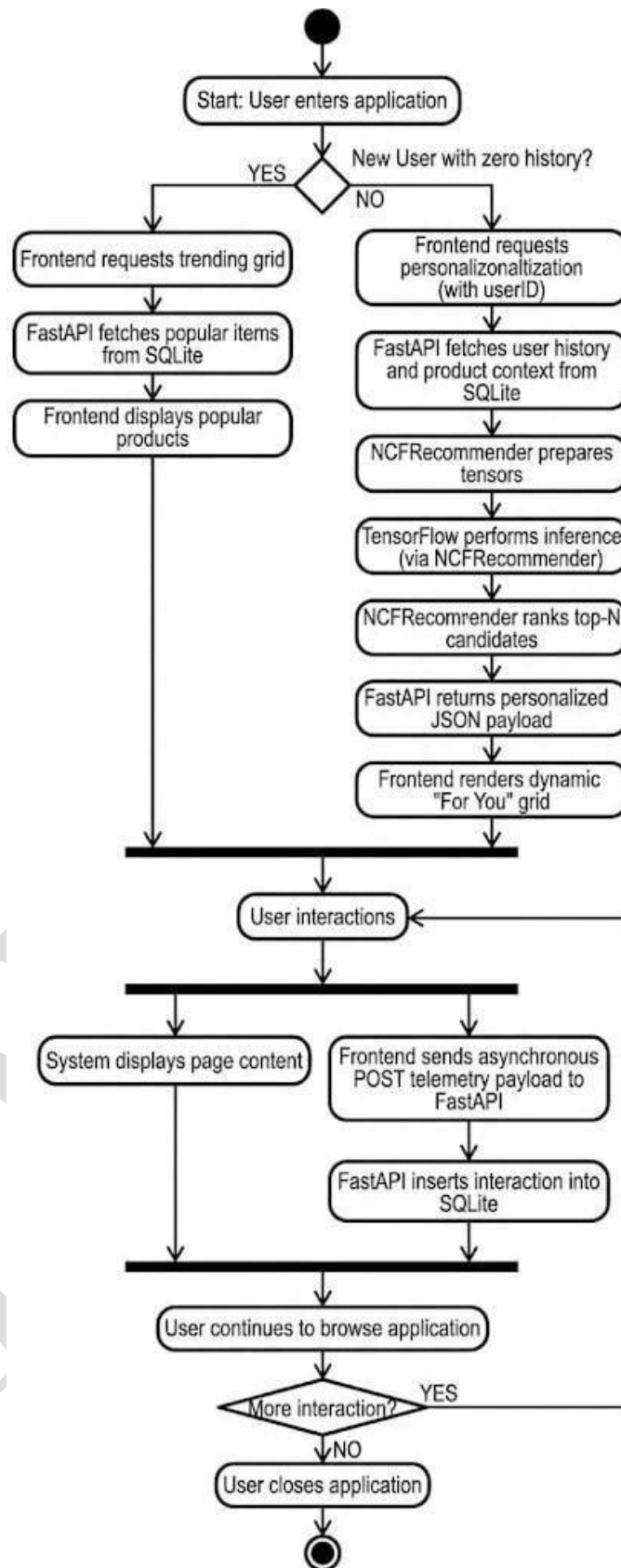


Fig. No. 5.4 Activity Diagram

5.5 SEQUENCE DIAGRAM

Fig. No. 5.5 describes the chronological flow of messages highlighting the asynchronous nature of the system: The Frontend App sends a GET request to the FastAPI Backend. The API queries the SQLite Database for user history and product metadata. The API forwards this context to the NCF Recommender class. The TensorFlow Engine runs the inference, generates similarity scores, and filters the top N candidates. The ranked list is sent back to the API, which returns a formatted JSON response to the Frontend App to be rendered.

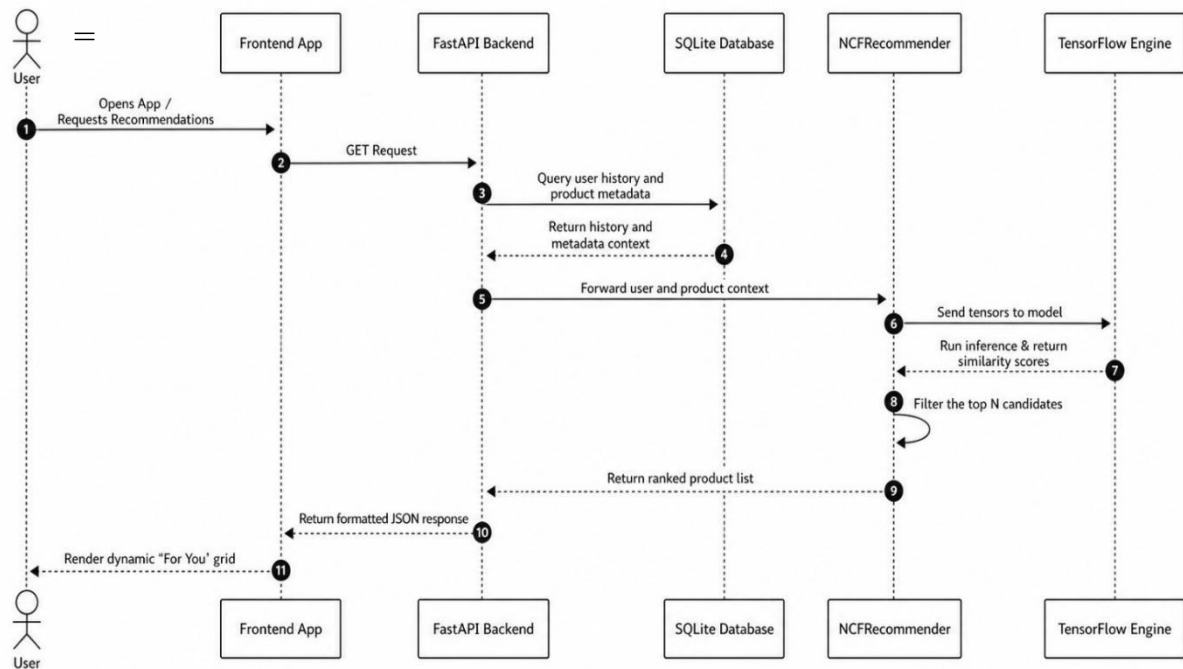


Fig. No. 5.5 Sequence Diagram

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 MODULES OF THE PROJECT

The Data-Driven Product Recommendation System (DDPRS) is structurally divided into distinct functional modules. This modular approach ensures a clean separation of concerns, isolating user interface operations, background telemetry logging, and heavy machine learning computations. By breaking the architecture down into these specific components, the application achieves high scalability, easier debugging, and seamless real-time performance. The system consists of the following 5 modules:

1. Catalog Management Module
2. Real-Time Telemetry Module
3. Cold Start Module
4. Neural Collaborative Filtering Module
5. Backend API Module

6.1.1 Catalog Management Module

This module is responsible for managing all product-related information efficiently. It handles the retrieval of items from the database, including product names, categories, prices, and global popularity scores. By providing dedicated API endpoints, it allows the frontend to fetch accurately paginated, filtered, and sorted product lists. This module plays a crucial role in ensuring the e-commerce catalog remains highly navigable and responsive when users browse different categories.

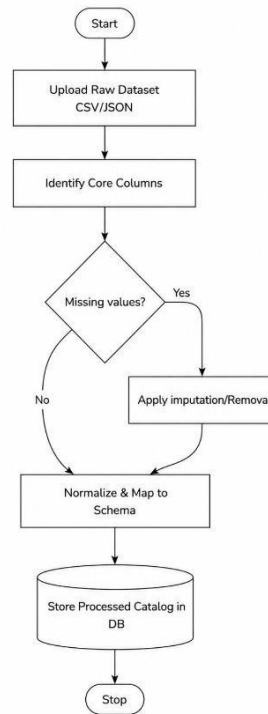


Fig. No. 6.1 Catalog Management Module Flow

Fig. No. 6.1 shows how the system queries the SQLite database based on frontend category filters and returns formatted JSON product lists.

Product Retrieval Algorithm

Algorithm

- Step 1: Start
- Step 2: Read category filter and limit
- Step 3: Query SQLite database for products
- Step 4: If products found, format as JSON
- Step 5: Else, return empty list
- Step 6: End

Pseudocode

START

INPUT category, limit

IF category is provided THEN

query = "SELECT * FROM products WHERE category = category"

```

ELSE
    query = "SELECT * FROM products"
END IF
EXECUTE QUERY
RETURN results
END

```

6.1.2 Real-Time Telemetry Module

This module allows the system to passively track user behavior. Whenever a user clicks on a product or views a category, an asynchronous POST request is sent to the backend containing the user ID, product ID, and interaction type. This data is securely appended to the SQLite database without interrupting the user's browsing experience. This historical data is essential for training and running the machine learning model.

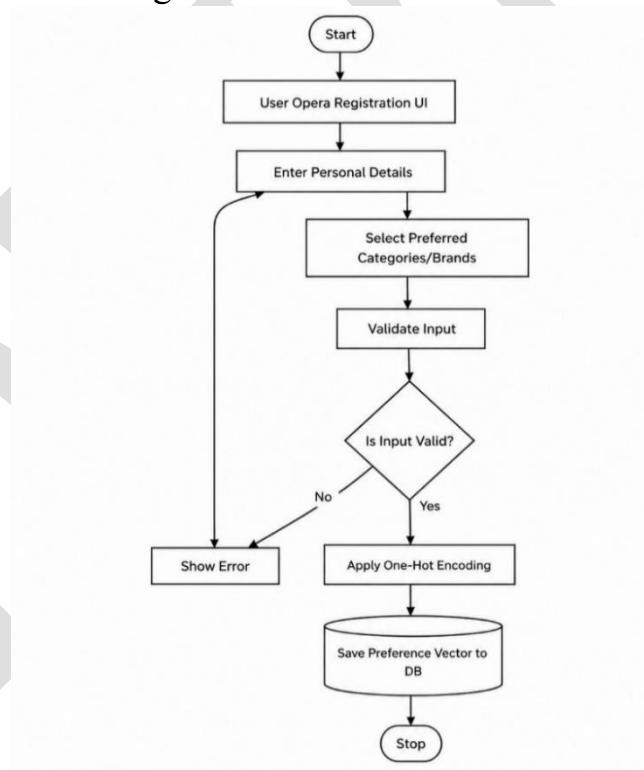


Fig .No. 6.2 Real-Time Telemetry Module Flow

Fig. No. 6.2 shows the working of the Real-Time Telemetry Module. It shows how JavaScript event listeners capture clicks and send tracking payloads

to the FastAPI backend for storage.

Telemetry Logging Algorithm

Algorithm

Step 1: Start

Step 2: Detect user click on product

Step 3: Extract UserID and ProductID

Step 4: Send POST request to backend API

Step 5: Insert record into SQLite database

Step 6: Stop

Pseudocode

BEGIN

ON product_click

READ userID, productID

payload = {userID, productID, action="view"}

SEND_ASYNC_REQUEST(payload)

SAVE data in database END

6.1.3 Cold Start Module

This module is the core component that handles brand new users who have zero interaction history. Instead of showing an empty page or random items, the system queries the database for the top trending or most popular products based on global interaction counts. This mechanism ensures that the user is immediately engaged with high-quality content.

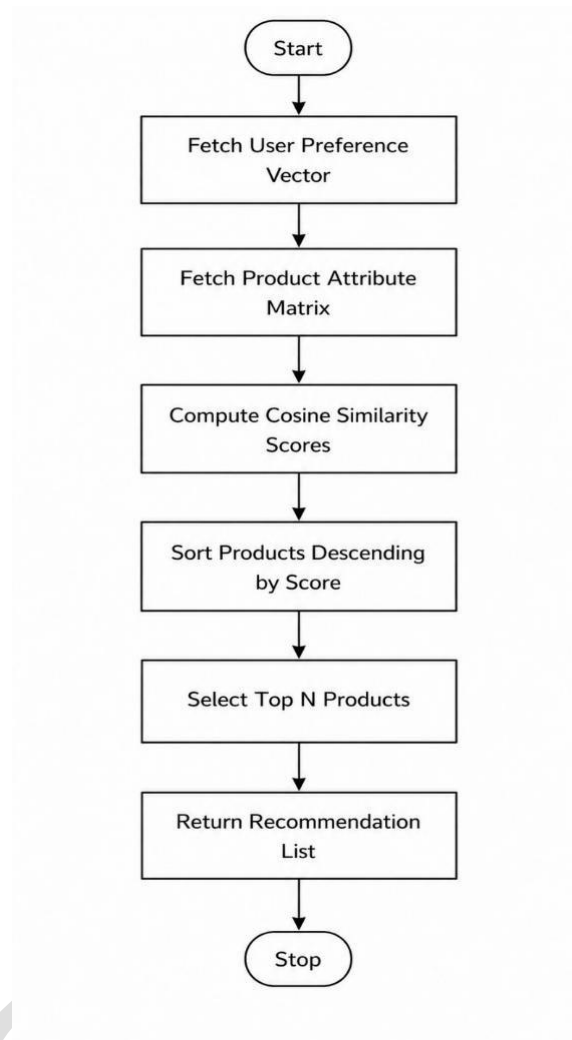


Fig .No. 6.3 Cold Start Module Flow

Fig. No. 6.3 shows the working of the Cold Start Module. It shows how the system checks the user's history length, and if it is zero, reroutes the recommendation request to the global popularity database query.

Popularity Fallback Algorithm

Algorithm

Step 1: Start

Step 2: Check user interaction history

Step 3: If history is empty, fetch popular products

Step 4: Sort products by interaction count

Step 5: Return top trending items

Step 6: End

Pseudocode

START

GET user_history

IF user_history IS EMPTY THEN

 trending_items = QUERY database FOR popular products

 RETURN trending_items

END IF

END

6.1.4 Neural Collaborative Filtering Module

This module facilitates the deep learning predictions. It loads the pre-trained TensorFlow model artifacts into the server memory. When an existing user requests recommendations, the system retrieves their specific interaction history, formats it into numerical tensors, and passes it through the multi-layer neural network to compute personalized similarity scores.

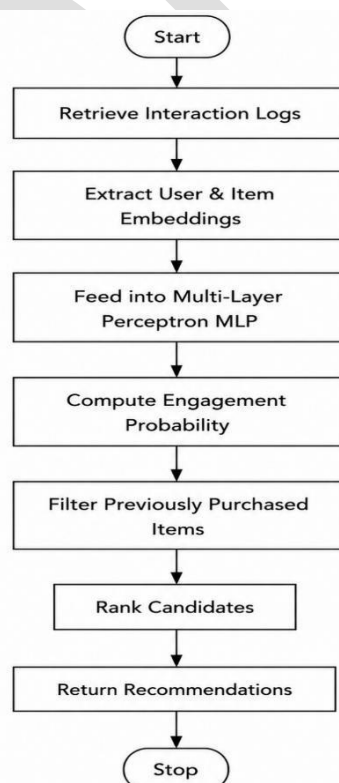


Fig .No. 6.4 Neural Collaborative Filtering Module Flow

Fig. No.6.4 shows the NCF Module. It shows how user data is fed into the Keras model, predictions are generated, and the top items are returned to the API.

6.1.5 Backend API Module

This module handles all server-side operations and acts as a bridge between the vanilla JavaScript frontend and the SQLite database. Built with FastAPI and running on Uvicorn, it processes incoming HTTP GET and POST requests asynchronously. The module manages data flow efficiently and ensures that the heavy TensorFlow computations do not block other concurrent users from browsing the store.

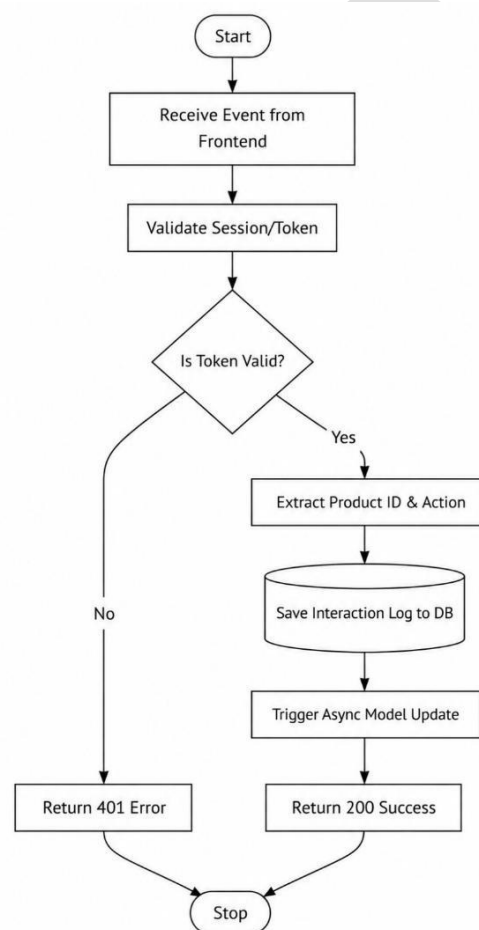


Fig .No. 6.5 Backend API Module Flow

Fig. No. 6.5 outlines the Backend API Module, bridging the frontend and database by routing requests, processing Python logic, and returning JSON.

CHAPTER 7

EXPERIMENTAL RESULTS

7.1 CATALOG BROWSING SCREENS

Fig. No. 7.1 and Fig. No. 7.2 show the catalog browsing screens of DDPRS, which introduce key features such as seamless product discovery, using clean grid layouts, clear product images, pricing details, and category filters to guide users before they click into specific items.

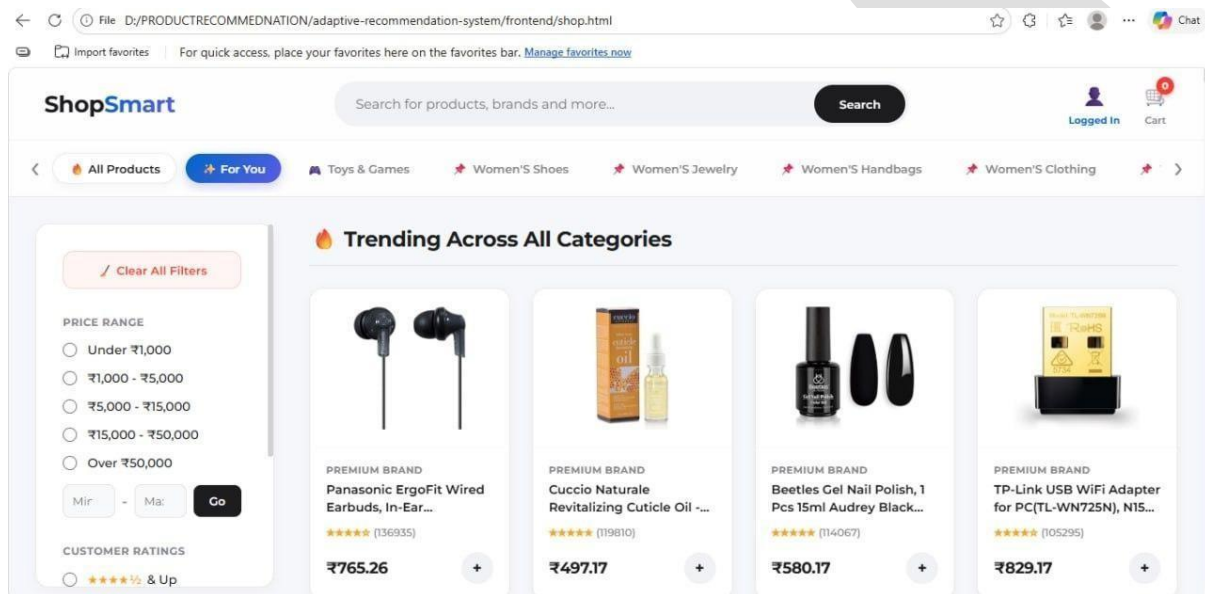


Fig. No. 7.1 Home Dashboard Screen

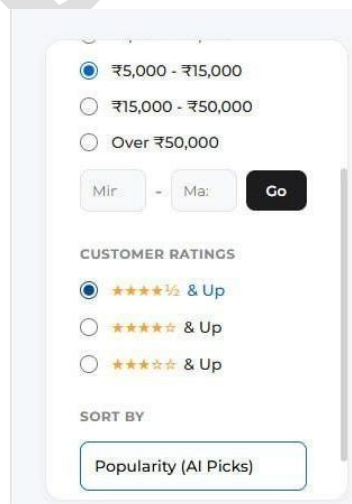
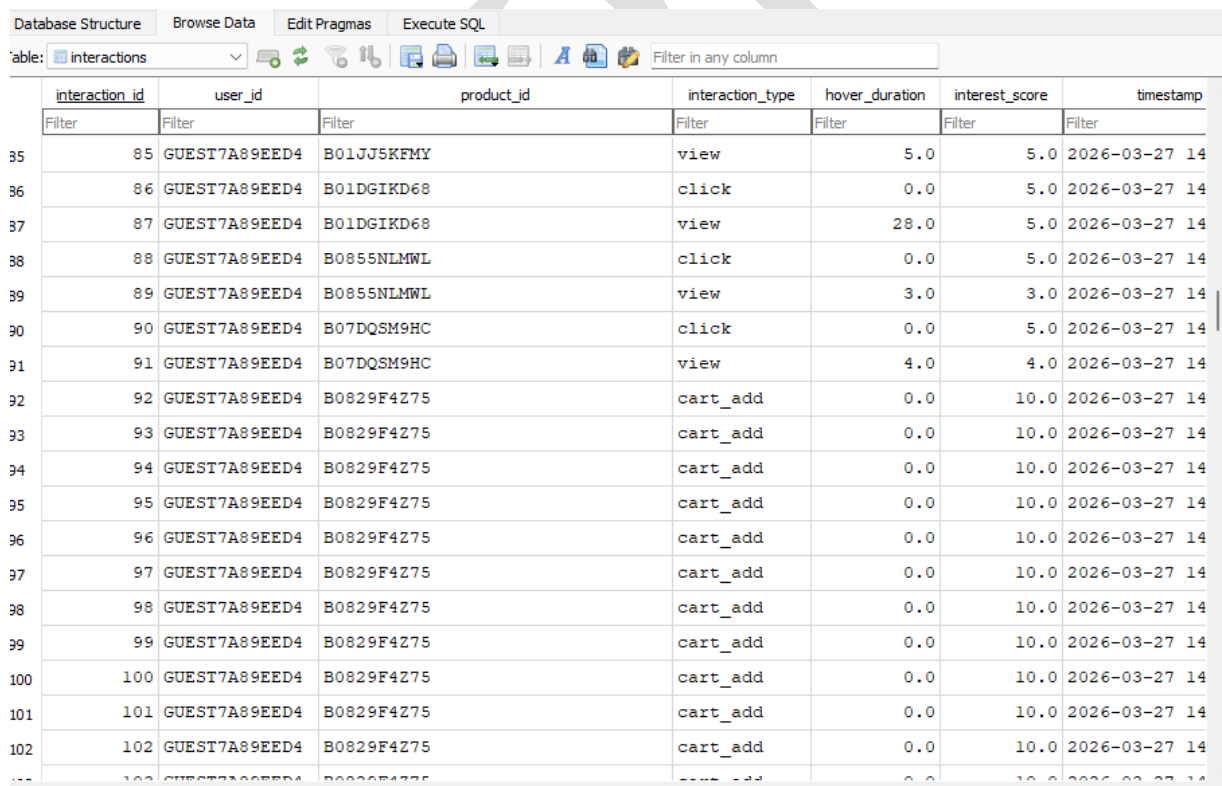


Fig. No. 7.2 Category Filter Screen

7.2 BACKEND DATA LOGGING AND SCORING

Fig. No. 7.3 displays the interaction telemetry table, capturing real-time behavioral data such as user IDs and product IDs. This table includes a calculated interest score for each event, which serves as the primary input for the recommendation engine by transforming raw actions into structured numerical weights. These values are then processed by the Neural Collaborative Filtering layers to identify latent user preferences.

Complementing this, Fig. No. 7.4 shows the global product catalog sorted by popularity score. This data forms the mathematical basis for the Cold Start module, allowing the system to serve high-quality trending items to brand-new users without existing history.



	interaction_id	user_id	product_id	interaction_type	hover_duration	interest_score	timestamp
	Filter	Filter	Filter	Filter	Filter	Filter	Filter
35	85	GUEST7A89EED4	B01JJ5KFMX	view	5.0	5.0	2026-03-27 14
36	86	GUEST7A89EED4	B01DGIKD68	click	0.0	5.0	2026-03-27 14
37	87	GUEST7A89EED4	B01DGIKD68	view	28.0	5.0	2026-03-27 14
38	88	GUEST7A89EED4	B0855NLMWL	click	0.0	5.0	2026-03-27 14
39	89	GUEST7A89EED4	B0855NLMWL	view	3.0	3.0	2026-03-27 14
30	90	GUEST7A89EED4	B07DQSM9HC	click	0.0	5.0	2026-03-27 14
31	91	GUEST7A89EED4	B07DQSM9HC	view	4.0	4.0	2026-03-27 14
32	92	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
33	93	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
34	94	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
35	95	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
36	96	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
37	97	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
38	98	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
39	99	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
100	100	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
101	101	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
102	102	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14
103	103	GUEST7A89EED4	B0829F4Z75	cart_add	0.0	10.0	2026-03-27 14

Fig. No. 7.3 Interaction telemetry table

1	SELECT product_id, name, category, popularity_score			
2	FROM products			
3	ORDER BY popularity_score DESC;			
4				

	product_id	name	category	popularity_score
1	B00DI88YCC	HDMI Cable 4K - 20ft - with A.I.S ...	Televisions & Video Products	712820.8
2	B00DI895MU	HDMI Cable 4K - 40ft - with A.I.S ...	Televisions & Video Products	712820.8
3	B09N3ZNHTY	BOAT Airdopes 141 Bluetooth Truly ...	Headphones & Earbuds	663191.2
4	B075XLWML4	Roku Streaming Stick+ HD/4K/HDR ...	Televisions & Video Products	589521.0
5	B003EM8008	Panasonic ErgoFit Wired Earbuds, In...	Headphones & Earbuds	588820.5
6	B071S6GZ1K	Men's Casual Slide	Men'S Shoes	565897.5
7	B0045PKGE6	Cuccio Naturale Revitalizing Cuticl...	Foot, Hand & Nail Care Products	563107.0
8	B09TVVGXWS	OnePlus Bullets Wireless Z2 ...	Headphones & Earbuds	533370.6
9	B08HYW2MKG	Beetles Gel Nail Polish, 1 Pcs 15ml...	Foot, Hand & Nail Care Products	513301.5
10	B07SJR6HL3	Soundcore Anker Life P2 True ...	Headphones & Earbuds	499098.6
11	B01M1C93ZF	Glad for Pets Black Charcoal Puppy ...	Cat Supplies	487741.5
12	B003N9M6YI	Nuby Ice Gel Teether Keys	Baby & Toddler Toys	487469.9
13	B00VPVEVSK	Colossal Foot rasp Foot File and ...	Foot, Hand & Nail Care Products	463702.0
14	B008IFXQFU	TP-Link USB WiFi Adapter for PC(TL-...	Televisions & Video Products	463298.0
15	B08DJBGD5L	Sengled Alexa Light Bulb, S1 Auto ...	Light Bulbs	460732.8
16	B07TLL6734	MRS. MEYER'S CLEANDAY Hand Soap, ...	Foot, Hand & Nail Care Products	444676.4

Fig. No. 7.4 Global product catalog

7.3 RECOMMENDATION SCREENS

Fig. No. 7.5 illustrates the "For You" personalized grid screen, which provides an overview of items specifically tailored by the NCF model based on past clicks. The backend achieves this by mapping the user's unique latent factors against the entire product embedding space to find the highest-ranking matches. In contrast, Figure 7.6 depicts the Cold Start version of this screen, displaying globally trending products for brand-new users with no history. This dual-path architecture ensures that the interface remains dynamic and engaging at all times. By seamlessly switching between personalized and popularity-based rankings, the system maintains high retention rates regardless of whether the user is a returning customer or a first-time visitor.

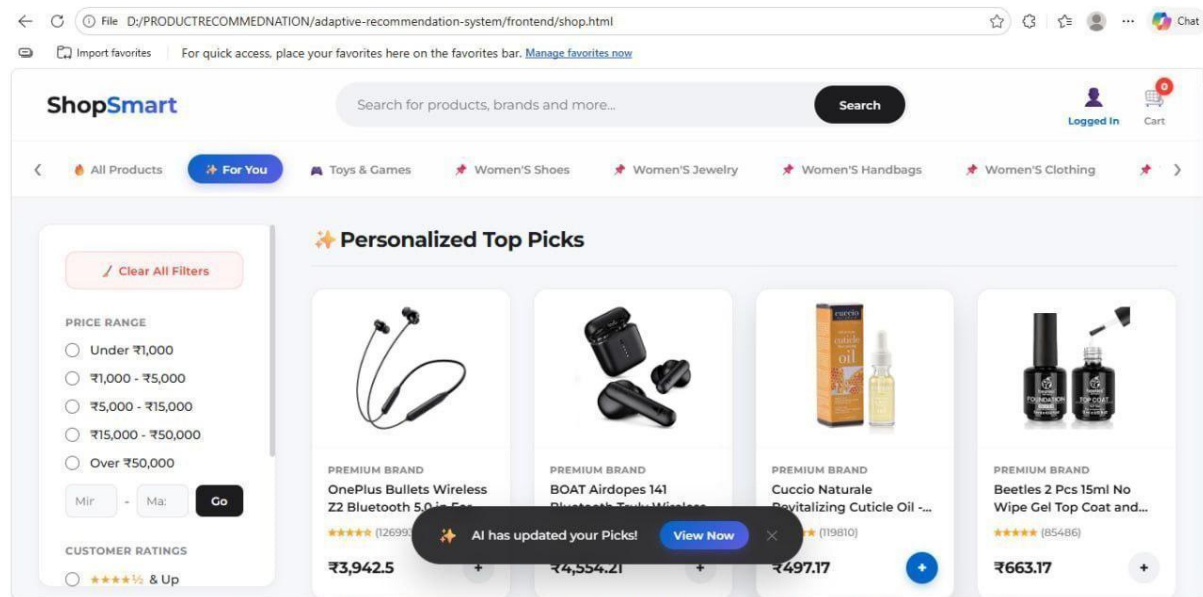


Fig. No. 7.5 Personalized Recommendations Screen

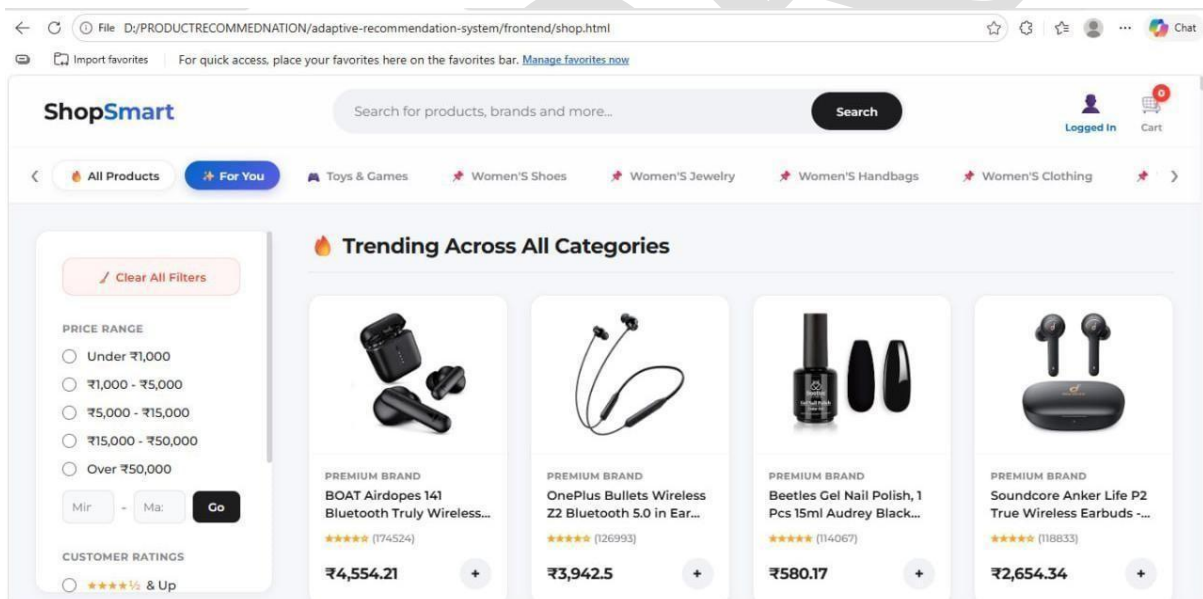


Fig. No. 7.6 Cold Start Popularity Screen

CHAPTER 8

RESULTS AND FINDINGS

8.1 PERFORMANCE EVALUATION

The proposed system is compared with existing synchronous, matrix factorization-based systems based on response time, recommendation accuracy, and cold start retention. The results show that the proposed system performs better by providing higher accuracy, faster load times, and improved user experience in a single platform.

8.1.1 Response Time Comparison

Fig. No. 8.1 shows the comparison of response time between the existing synchronous system and the proposed asynchronous FastAPI system. The proposed system performs faster in operations such as loading product categories and submitting telemetry logs without freezing the UI.

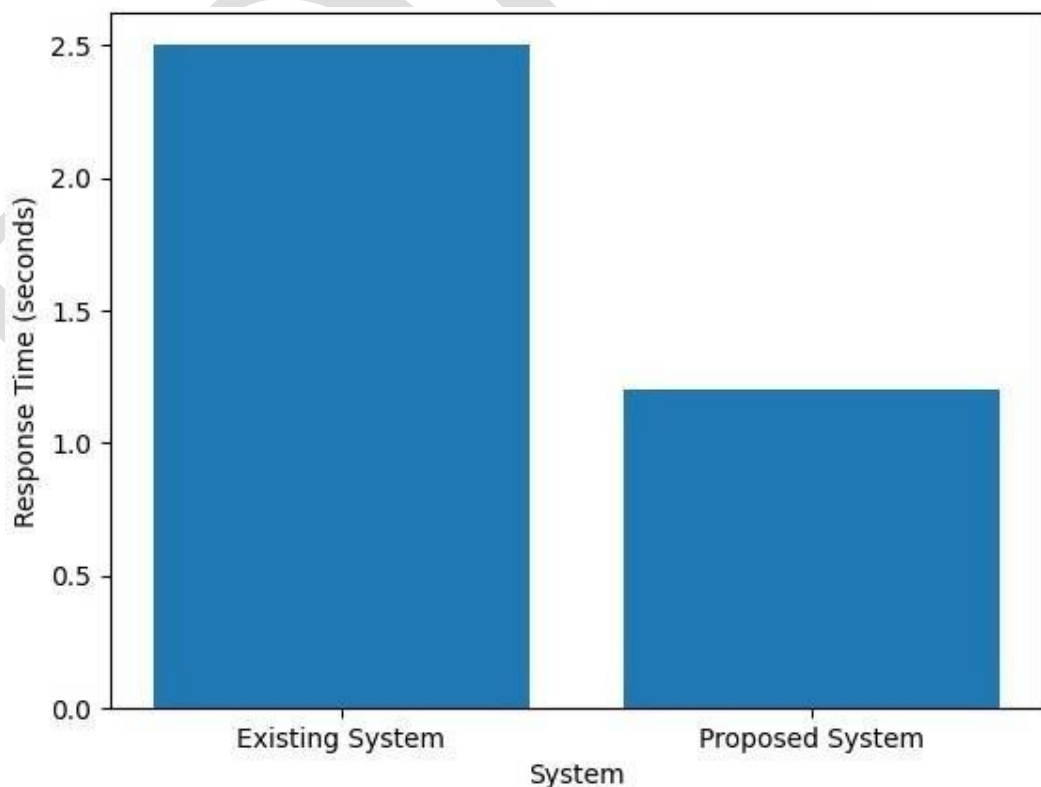


Fig. No. 8.1 Response Time Comparison

8.1.2 Recommendation Accuracy Comparison

Fig. No. 8.2 illustrates the prediction accuracy comparison between standard collaborative filtering and the Neural Collaborative Filtering (NCF) model. The proposed NCF system shows a significantly lower error rate because it successfully maps non-linear behavioral patterns.

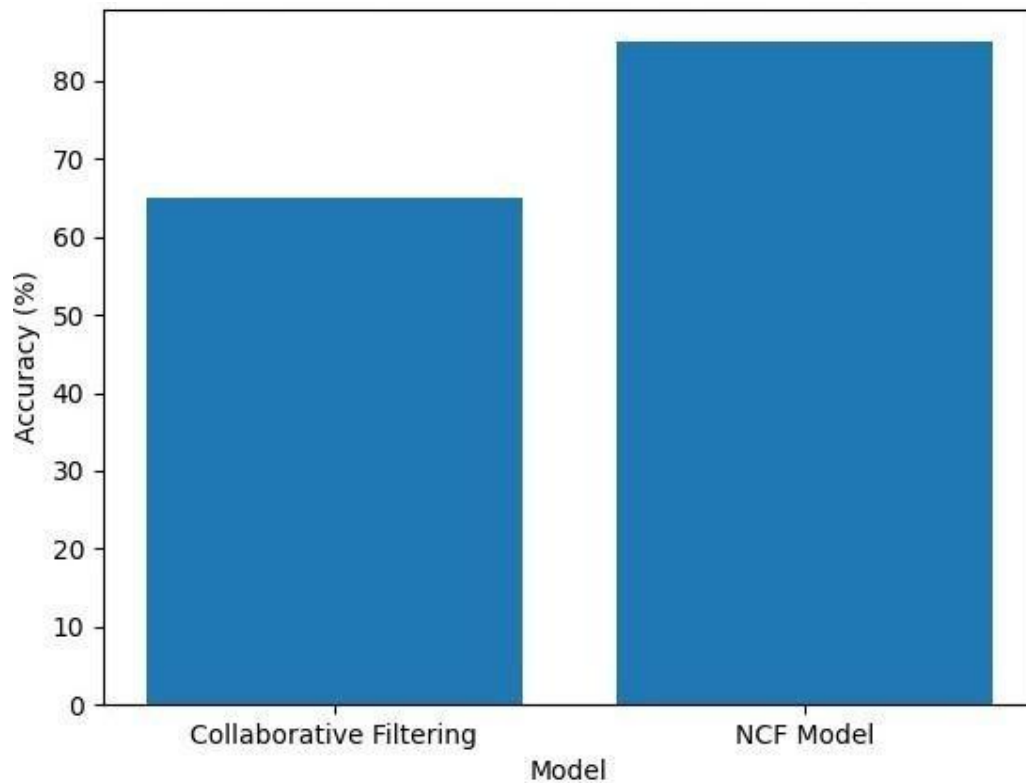


Fig. No. 8.2 Recommendation Accuracy Comparison

8.1.3 Cold Start User Engagement Comparison

Fig. No. 8.3 shows the comparison of user retention during their first visit. Existing systems show a high drop-off rate due to generic or empty screens, while the proposed system maintains high engagement by instantly providing trending products as a fallback.

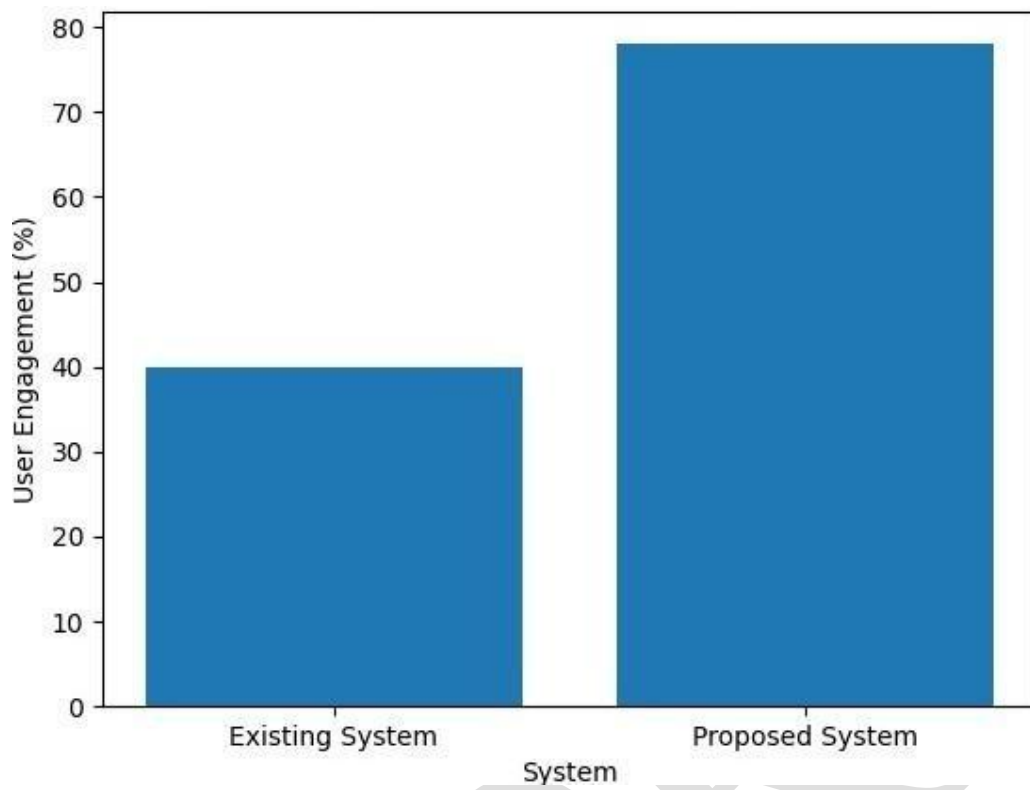


Fig. No. 8.3 Cold Start Engagement Comparison

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

The Data Driven Product Recommendation System (DDPRS) is a comprehensive, user-friendly, and efficient solution developed to provide highly personalized e-commerce experiences. It effectively solves the cold start problem by intelligently displaying popular items while actively learning from real-time behavioral telemetry. The system integrates a robust deep learning model with a modern, asynchronous FastAPI backend, ensuring that heavy computations do not slow down the user's browsing experience. By bringing these services into a single platform, the application reduces latency, ensures high-speed data retrieval, and enhances the overall shopping experience. Overall, the system contributes to improved product discovery, higher customer retention, and a more technologically advanced approach to digital retail personalization.

9.2 FUTURE ENHANCEMENTS

In future, we plan to implement a dynamic pipeline for real-time model retraining, allowing the NCF model to update its weights instantly as new telemetry arrives. Additionally, the system can be expanded into a hybrid model by incorporating Content-Based filtering using natural language processing on product descriptions, and integration with cloud-based clusters for massive scalability.

APPENDIX (SOURCE CODE)

ncf_recommender.py

```
def build_model(self, num_users, num_items):  
    """Build Neural Collaborative Filtering architecture"""  
    user_input = layers.Input(shape=(1,), name='user_input')  
    item_input = layers.Input(shape=(1,), name='item_input')  
  
    # Embedding layers with L2 regularization  
    user_embedding = layers.Embedding(  
        num_users,  
        self.embedding_dim,  
        embeddings_regularizer=regularizers.l2(1e-6),  
        name='user_embedding'  
    )(user_input)  
  
    item_embedding = layers.Embedding(  
        num_items,  
        self.embedding_dim,  
        embeddings_regularizer=regularizers.l2(1e-  
        6), name='item_embedding'  
    )(item_input)  
  
    user_vec = layers.Flatten()(user_embedding) # Flatten embeddings  
    item_vec = layers.Flatten()(item_embedding)  
  
    concat = layers.Concatenate()([user_vec, item_vec]) x = concat # Deep neural  
    network layers  
    for i, units in  
        enumerate(self.hidden_layers): x =
```

```

layers.Dense(
    units,
    activation='relu'
    ,
    kernel_regularizer=regularizers.l2(1e-5),
    name=f'dense_{i}'
)(x)
x =
layers.BatchNormalization()(x) x
= layers.Dropout(0.3)(x)

# Output layer (probability of interaction)
output = layers.Dense(1, activation='sigmoid', name='output')(x)

# Build model
model = Model(inputs=[user_input, item_input], outputs=output)

```

main.py (FastAPI)

```

from fastapi import FastAPI, HTTPException, Depends, Header
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import sqlite3
import logging
import sys
from fastapi.staticfiles import StaticFiles
from ncf_recommender import NCFRecommender
ncf_model = NCFRecommender()#NCFMODEL
try:
    ncf_model.load('models/saved')

```

```

logger.info(" NCF Deep Learning model loaded!")

@app.get = FastAPI(title="ShopSmart API", version="2.0.0")#FastAPI
app.mount("/static", StaticFiles(directory="static"), name="static")

@app.get("/recommend/cold-start/{session_id}")
async def cold_start_recommendations(session_id: str, limit: int =60): try:
    conn = get_db()
    conn.row_factory =
    sqlite3.Row cursor =
    conn.cursor() cursor.execute("""
        SELECT p.category, p.name, p.brand, p.price, i.interest_score
        FROM interactions i
        JOIN products p ON i.product_id = p.product_id
        WHERE i.user_id = ?
        ORDER BY i.timestamp DESC
        LIMIT 15
    """, (session_id,))
    recent_items =
    cursor.fetchall() if
    recent_items:
        category_scores =
        {} prices = []
        keywords_seen = []
if __name__ == "__main__
    _: import uvicorn
    print("\n" + "="*60 + " STARTING API v2.0\n" + "="*60)
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True

```

REFERENCES

- [1] Xu, K., Zhou, H., Zheng, H., Zhu, M., and Xin, Q. (2024) ‘Intelligent classification and personalized recommendation of E-commerce products based on machine learning’, *Applied and Computational Engineering*, Vol. 64, pp. 143–149.
- [2] Li, S. (2024) ‘Harnessing Multimodal Data and Mult-Recall Strategies for Enhanced Product Recommendation in E-Commerce’, DOI: 10.20944/preprints202409.2417.v1.
- [3] Xiang, A., Huang, B., Guo, X., Yang, H., and Zheng, T. (2024) ‘A Neural Matrix Decomposition Recommender System Model based on the Multimodal Large Language Model’, pp. 146–150, DOI: 10.1145/3696271.3696295.
- [4] Shokrzadeh, Z., Feizi Derakhshi, M. R., Balafar, M., and Bagherzadeh, J. (2023) ‘Knowledge graph-based recommendation system enhanced by neural collaborative filtering and knowledge graph embedding’, *Ain Shams Engineering Journal*, Vol. 15, pp. 102263.
- [5] Xu, W., Xiao, J., and Chen, J. (2024) ‘Leveraging Large Language Models to Enhance Personalized Recommendations in E-commerce’, pp. 1–6, DOI: 10.1109/ICECCE63537.2024.10823618.
- [6] Zhao, Z., Zhang, N., Xiong, J., Feng, M., Jiang, C., and Wang, X. (2024) ‘Enhancing E-commerce Recommendations: Unveiling Insights from Customer Reviews with BERTFusionDNN’, *Journal of Theory and Practice of Engineering Science*, Vol. 4, No. 2.

- [7] Chen, W., Shen, Z., Pan, Y., Tan, K., and Wang, C. (2024) ‘Applying Machine Learning Algorithm to Optimize Personalized Education Recommendation System’, *Journal of Theory and Practice of Engineering Science*, Vol. 4, No. 1.
- [8] Hasan, M. R., and Ferdous, J. (2024) ‘Dominance of AI and Machine Learning Techniques in Hybrid Movie Recommendation System Applying Text-to-number Conversion and Cosine Similarity Approaches’, *Journal of Computer Science and Technology Studies*, Vol. 6, No. 1, pp. 94–102.
- [9] Shang, F., Zhao, F., Zhang, M., Sun, J., and Shi, J. (2024) ‘Personalized Recommendation Systems Powered By Large Language Models: Integrating Semantic Understanding and User Preferences’, *International Journal of Innovative Research in Engineering and Management (IJIREM)*, Vol. 11, No. 4, pp. 39–49.
- [10] Ruan, T., Liu, Q., and Chang, Y. (2025) ‘Digital media recommendation system design based on user behavior analysis and emotional feature extraction’, *PLoS One*, Vol. 20, No. 2, pp. e0322768.



Date: **APRIL 2026**

CERTIFICATE

This is to certify that **SANJAY S.S (Reg No: 962822205034), NEELAKANDAN S (Reg No: 962822205301), ABHILASH A.S (Reg No: 962822205304), BEZALEL VIVIAN SALES S (Reg No: 962822205306)**, students of **B.Tech Information Technology, University College Of Engineering, Nagercoil** have successfully completed their project entitled **"DATA-DRIVEN PRODUCT RECOMMENDATION SYSTEM"** at our firm. They have been fully involved in this project from **09 January 2025 to 09 April 2026** and have done considerable research both on the subject of the project as well as the environment in which they have developed the software.

Their work has been evaluated and tested at every stage and we glad to certify that they have produced a useful Website application.

Ms. Varsha V Nair

Academic Project Coordinator



Mr. Renjith Raju

Head of Academic Department